1. Project Configuration: pom.xml

The pom.xml (Project Object Model) is the XML file that contains information about the project and configuration details used by Maven to build the project.

Why is it used?

It serves as the "manifesto" for the project, defining dependencies, build settings, and the Java version, ensuring that the development environment is consistent for anyone working on the code.

Key Components Breakdown:

***Parent POM (spring-boot-starter-parent)**:*

***What**: Acts as a template providing default configurations for Spring Boot applications.*

***Why**: It manages versions for all internal dependencies (like Spring Security or JPA) so you don't have to specify them manually, preventing version conflicts.*

***Java Version (17)**:*

***What**: Sets the source and target compatibility to Java 17.*

***Why**: To utilize modern Java features like Records and improved performance.*

***Core Dependencies**:*

***spring-boot-starter-data-jpa**: Allows the app to interact with the MySQL database using Java objects (Hibernate).*

***spring-boot-starter-security**: Provides the framework for authentication and authorization.*

***java-jwt (v4.4.0)**: A library used specifically to handle JSON Web Tokens for secure communication.*

***mysql-connector-j**: The driver required to connect the Java application to a MySQL database.*

***lombok**: A library that automatically generates boilerplate code like Getters, Setters, and Constructors at compile time.*

2. Application Entry Point: UserServiceApplication.java

This is the main class that launches the Spring Boot application.

Why is it used?

It serves as the starting point of the JVM (Java Virtual Machine) and handles the initial data "seeding" (filling the database with starting values) so the application is ready to use immediately upon startup.

Key Code Blocks Breakdown:

***@SpringBootApplication**:*

***What**: A convenience annotation that combines @Configuration, @EnableAutoConfiguration, and @ComponentScan.*

***Why**: It tells Spring Boot to look for other components (Controllers, Services) in the current package and set up the application context automatically.*

***public static void main(String[] args)**:*

***What**: The standard Java entry point.*

***Why**: It calls SpringApplication.run, which starts the embedded Tomcat server and initializes the entire Spring framework.*

***@Bean CommandLineRunner run(UserService userService)**:*

***What**: A functional interface used to run a specific piece of code once the application context is fully loaded.*

***Why**:*

***Role Seeding**: It programmatically creates roles like ROLE_USER, ROLE_ADMIN, and ROLE_SUPER_ADMIN in the database.*

***User Seeding**: It creates initial users (e.g., "John Traore", "Will Smith") and saves them using the userService.*

***Relationship Mapping**: It assigns roles to these users (e.g., assigning ROLE_SUPER_ADMIN to "arnold") to test authorization logic.*

***@Bean PasswordEncoder passwordEncoder()**:*

***What**: Defines a BCryptPasswordEncoder as a Spring Bean.*

***Why**: To ensure that user passwords are not stored in plain text. By defining it here, it can be "Injected" into any other class (like UserServiceImpl) to hash passwords before saving them to the database.*

1. The Role Model: Role.java

*The **Role** entity represents the different levels of authority within the system (e.g., **ROLE_USER**, **ROLE_ADMIN**).*

Why is it used?

*In security-focused applications, it is better to separate users from their permissions. By having a dedicated **Role** entity, you can assign, remove, or update permissions globally without modifying every individual user record.*

Key Components Breakdown:

***@Entity**:*

***What**: A JPA (Java Persistence API) annotation.*

***Why**: It tells the application that this class should be mapped to a table in the database named "Role".*

***@Data, @NoArgsConstructor, @AllArgsConstructor**:*

***What**: Lombok annotations.*

***Why**: These eliminate "boilerplate" code by automatically generating getters, setters, and constructors, keeping the file clean and readable.*

***@Id and @GeneratedValue**:*

***What**: Defines the Primary Key.*

***Why**: GenerationType.AUTO allows the database to automatically increment the ID number whenever a new role is created.*

2. The User Model: User.java

*The **User** entity represents the people who can log into the system.*

Why is it used?

*It acts as the central data structure for authentication, storing the credentials (**userName** and **password**) and the associated permissions for each person.*

Key Components Breakdown:

***private Collection<Role> roles**:*

***What**: A collection (list) that holds Role objects.*

***Why**: This allows a single user to hold multiple roles simultaneously (e.g., a user can be both an "Admin" and a "Manager").*

***@ManyToMany(fetch = FetchType.EAGER)**:*

***What**: Defines the relationship between Users and Roles.*

***Why**:*

***ManyToMany**: In the real world, many users can share the same role, and one user can have many roles.*

***FetchType.EAGER**: This ensures that whenever a User is loaded from the database, all their associated Roles are loaded immediately. This is critical for security checks where you need to know a user's permissions the moment they log in.*

***new ArrayList<>() initialization**:*

***What**: Initializes the roles list.*

***Why**: This prevents a NullPointerException if you try to add a role to a brand-new user before the list has been created.*

1. Data Access Layer: UserRepo.java & RoleRepo.java

These interfaces are the bridge between the Java application and the MySQL database using Spring Data JPA.

Why are they used?

They allow you to perform CRUD (Create, Read, Update, Delete) operations without writing complex SQL queries manually.

Key Components Breakdown:

***JpaRepository<Entity, IdType>**:*

***What**: By extending this, the interfaces automatically inherit methods like save(), findAll(), and deleteById().*

***Custom Query Methods**:*

***findByUserName(String username)**: A dynamic finder method that JPA translates into SELECT * FROM User WHERE user_name = ?.*

***findByName(String name)**: Similarly retrieves a specific role by its string name (e.g., "ROLE_ADMIN").*

2. Service Layer: UserService.java & UserServiceImpl.java

The Service Layer contains the core business logic. It coordinates the repositories and security configurations.

Why is it used?

It provides a clean API for the controller to use, ensuring that logic like password hashing or role assignment happens consistently in one place.

Key Logic Breakdown in UserServiceImpl:

***@Service and @Transactional**:*

***What**: @Service marks it as a Spring-managed bean. @Transactional ensures that any database operation within a method either succeeds entirely or rolls back on error.*

***Benefit**: In addRoleToUser, we don't need to call repo.save() because the @Transactional annotation automatically detects changes to the "User" object and saves them at the end of the method.*

***loadUserByUsername (Security Integration)**:*

***What**: Implements the UserDetailsService interface required by Spring Security.*

***Why**: When a user tries to log in, Spring Security calls this method to fetch the user from the database and convert their Role entities into SimpleGrantedAuthority objects, which the security framework understands.*

***saveUser with Password Encoding**:*

***What**: Before saving a user, it calls passwordEncoder.encode().*

***Why**: This ensures that even if the database is compromised, the actual passwords are encrypted and unreadable.*

***@RequiredArgsConstructor**:*

***What**: A Lombok annotation that creates a constructor for all final fields.*

***Why**: It performs "Constructor Injection," which is the recommended way to link UserRepo, RoleRepo, and PasswordEncoder into the service.*

1. Data Transfer Objects: RoleToUserForm.java

A DTO is a simple object used to pass data between the software's layers or over a network.

Why is it used?

When a client sends a request to "Add a Role to a User," they don't send the entire User object. They only send two strings: the username and the roleName. Using a DTO prevents exposing the entire database entity structure to the public API and simplifies the request body.

Key Components Breakdown:

***Fields**: Contains only username and roleName.*

***Lombok @Data**: Automatically provides the getters and setters so Spring can map the incoming JSON request directly into this Java object.*

2. The API Controller: UserController.java

The Controller is the "Front Door" of your application. It handles incoming HTTP requests and returns responses.

Why is it used?

It acts as a traffic controller, taking data from the web (JSON), passing it to the UserService for processing, and then sending back a status code (like 200 OK or 201 Created).

Key Endpoints Breakdown:

@GetMapping("/users"):

***What**: Fetches all users.*

***Response**: Returns ResponseEntity.ok(), which translates to an HTTP 200 status with the list of users in the body.*

@PostMapping("/user/save"):

***What**: Creates a new user.*

***Why URI?**: It uses ServletUriComponentsBuilder to create a "Location" header. This is a REST best practice that tells the client exactly where the newly created resource can be found.*

@PostMapping("/role/addtouser"):

***What**: Maps a role to a user.*

***Logic**: It accepts the RoleToUserForm DTO and calls userService.addRoleToUser().*

***@RequiredArgsConstructor**:*

***What**: Injects the UserService bean.*

> ***Why**: This allows the controller to delegate all heavy lifting and database logic to the Service layer, keeping the controller thin and focused only on web communication.*

1. The Authentication Filter: CustomAuthenticationFilter.java

This class handles the login process. It intercepts the user's credentials and, if valid, issues a "passport" in the form of a JWT.

Why is it used?

Standard Spring Security uses sessions (cookies), which don't work well for mobile apps or distributed microservices. This filter replaces sessions with ***Stateless JWTs****.*

Key Logic Breakdown:

attemptAuthentication*:*

Action*: Extracts the username and password from the HTTP request.*

Process*: It wraps these into a UsernamePasswordAuthenticationToken and hands it to the AuthenticationManager to verify against the database.*

successfulAuthentication*:*

Action*: Runs only if the password is correct.*

JWT Creation*: It generates two tokens using the* ***HMAC256*** *algorithm:*

Access Token*: Valid for 10 minutes. Contains the user's roles (claims).*

Refresh Token*: Valid for 30 days. Used to get a new access token without re-typing a password.*

Response*: Sends the tokens back to the client as a JSON object.*

2. The Authorization Filter: CustomAuthorizationFilter.java

This class acts as a "security guard" for every single request coming into the API after the user has logged in.

Why is it used?

Since the server doesn't "remember" logged-in users (stateless), this filter checks every request for a valid token in the header.

Key Logic Breakdown:

Token Extraction*: It looks for a header named Authorization that starts with the word Bearer .*

Verification*: It uses the secret key to verify that the token hasn't been tampered with or expired.*

Security Context*: Once verified, it extracts the roles from the token and tells Spring Security: "This is 'John', and he has 'ROLE_ADMIN' permissions for this specific request".*

Error Handling*: If the token is fake or expired, it immediately sends a 403 Forbidden response and stops the request.*

3. Security Configuration: SecurityConfig.java

This is the master brain of the security system. It stitches all the previous pieces together.

Why is it used?

It defines the high-level security rules: which URLs are public, which require a login, and which roles can perform specific actions.

Key Configurations Breakdown:

Stateless Policy*: SessionCreationPolicy.STATELESS tells Spring NOT to create sessions on the server.*

Role-Based Access Control (RBAC)*:*

.permitAll()*: Allows everyone to access the /login and /token/refresh endpoints.*

.hasAnyAuthority("ROLE_USER")*: Restricts certain GET requests to users with the "USER" role.*

.hasAnyAuthority("ROLE_ADMIN")*: Restricts the ability to save new users only to Administrators.*

Filter Registration*: It explicitly tells Spring to run the CustomAuthorizationFilter before the standard authentication filters.*
DaoAuthenticationProvider*: This connects the UserDetailsService (which fetches users from the database) with the PasswordEncoder (which checks hashed passwords).*

1. External Configuration: application.properties
This file contains the environmental settings that Spring Boot uses to connect to external resources like the database.
Why is it used?
It separates the environment-specific data (like database passwords) from the Java code. This allows you to change the database location without recompiling the entire application.
Key Settings Breakdown:
***spring.datasource.url**: Specifies the connection string for the MySQL database (e.g., jdbc:mysql://localhost:3306/user_db).*
***spring.jpa.hibernate.ddl-auto=update**:*
***What**: Tells Hibernate how to manage the database schema.*
***Why**: The update setting ensures that if you add a new field to your User entity, Hibernate will automatically add that column to the MySQL table without deleting existing data.*
***spring.jpa.show-sql=true**:*
***What**: Prints every SQL query generated by JPA to the console.*
***Why**: Essential for debugging and optimizing performance during development.*
2. Password Configuration: PasswordConfig.java
While the password encoder was initially defined in the main application class, moving it to a dedicated config class is a best practice for modularity.
Why is it used?
It isolates security-related bean definitions, making the code easier to test and modify (for example, if you wanted to switch from BCrypt to Argon2 in the future).
Key Component:
***BCryptPasswordEncoder**:*
***What**: A strong hashing function that incorporates a "salt" to protect against rainbow table attacks.*
***Why**: It is the industry standard for securing user passwords in Spring applications.*
3. Custom Exception Handling: RoleNotFoundException.java
This is a custom Java class that extends RuntimeException.
Why is it used?
Generic errors (like NullPointerException) are hard to debug. A custom exception like RoleNotFoundException tells the developer exactly what went wrong: the system tried to assign a role that doesn't exist in the database.
Key Logic Breakdown:
***super(message)**:*
***What**: Passes the error message to the parent RuntimeException class.*
***Benefit**: When this error is thrown, the logs will clearly state: "Role 'MANAGER' not found" instead of a generic system crash, allowing for faster troubleshooting.*
4. Data Transfer: RoleToUserForm.java
This is a simple "Plain Old Java Object" (POJO) used specifically for web requests.

Why is it used?
When the frontend sends a request to link a user to a role, it sends a JSON object. This class acts as the "template" that Spring uses to read that JSON.
Key Components:
***username and roleName**: These are the only two pieces of data required to create a relationship between the two entities.*
***Lombok @Data**: Automatically generates the getters and setters needed by the Jackson library to map JSON fields to Java fields.*

1. Token Refresh Logic: UserController.java (Continued)
*In a secure JWT system, access tokens are short-lived. The **refreshToken** method handles the specialized logic for extending a user's session without requiring them to log in again.*
Why is it used?
It provides a balance between security and user experience. If an access token is stolen, it expires quickly. The refresh token allows the legitimate user to stay logged in securely for a longer duration.
Key Logic Breakdown:
***Header Check**: It looks for the Authorization header specifically starting with Bearer .*
***Token Verification**: It decodes the Refresh Token to ensure it is valid and hasn't expired.*
***User Lookup**: It uses the username from the refresh token to pull the latest user data (and roles) from the database via userService.getUser(username).*
***New Token Generation**: It issues a brand-new **Access Token** while keeping the original Refresh Token.*
***Error Handling**: If the refresh token is invalid or missing, it returns a 403 Forbidden status with a JSON error message.*
2. Password Security Configuration: PasswordConfig.java
This class is a dedicated configuration for handling sensitive credential encryption.
Why is it used?
*By isolating the **PasswordEncoder** in its own **@Configuration** class, the application ensures that the encryption logic is centralized and can be easily swapped or updated (e.g., increasing the strength of the hashing) without touching business logic.*
Key Component:
***BCryptPasswordEncoder Bean**:*
***What**: An implementation of the PasswordEncoder interface.*
***Function**: It uses a "salted" hashing algorithm. Every time a password is saved, BCrypt generates a random salt so that the same password (e.g., "password123") results in a different hash in the database, preventing "Rainbow Table" attacks.*
3. Application Integrity: UserServiceApplicationTests.java
This file contains the automated test suite for the application.
Why is it used?
It serves as a "health check" for the project. Before deploying the code to a server, these tests run to ensure that all Spring Beans (Services, Repositories, Controllers) load correctly and that the database connection is functional.
Key Components Breakdown:
***@SpringBootTest**:*
***What**: Tells Spring to bootstrap the full application context for the test.*
***Why**: It ensures the integration between all layers (Web, Service, Data) works as expected.*
***contextLoads()**:*
***What**: A smoke test.*
***Why**: If this method fails, it means the application has a critical configuration error (like a wrong database password or a missing dependency) that would prevent it from starting in production.*

1. Custom Exception Handling: RoleNotFoundException.java

*This is a custom Java class that extends **RuntimeException** to handle specific business logic failures.*

Why is it used?

*Standard Java exceptions (like **NullPointerException**) are too vague for debugging complex systems. By creating a **RoleNotFoundException**, the developer can pinpoint exactly why a process failed—specifically that a user tried to assign or access a role that does not exist in the database.*

Key Components Breakdown:

***extends RuntimeException**:*

***What**: Inherits from the standard unchecked exception class in Java.*

***Why**: It allows the application to stop the current execution flow and trigger an error response without requiring "try-catch" blocks everywhere in the code.*

***super(message)**:*

***What**: Passes a custom error string to the parent class.*

***Why**: When this error is thrown, the server logs will show a human-readable message (e.g., "Error: Role 'SUPER_ADMIN' not found"), making it much faster to troubleshoot database inconsistencies.*

2. Data Transfer Objects (DTO): RoleToUserForm.java

A DTO is a simple object used specifically to transport data between the client (like a mobile app) and the server.

Why is it used?

*When adding a role to a user, the frontend shouldn't have to send the entire **User** and **Role** database objects. This DTO simplifies the request by only requiring the essential "keys" (the names), which improves performance and security.*

Key Components Breakdown:

***private String username**: The unique identifier for the user receiving the role.*

***private String roleName**: The name of the role being assigned.*

***Lombok @Data**:*

***What**: Automatically generates getters, setters, and toString methods.*

***Why**: It keeps the class extremely small and readable while allowing Spring's ObjectMapper to automatically fill these fields from an incoming JSON request.*

3. Resource Management: application.properties

This file contains the configuration settings that define how the application connects to the outside world.

Why is it used?

It allows the application to be "environment-aware." You can change the database URL or the server port here without ever touching the Java source code.

Key Settings (Internal Defaults):

***Database Connection**: Defines the driver, username, and password for the MySQL instance.*

***Hibernate DDL**: Typically set to update or create, which tells the system to automatically build the database tables based on your User.java and Role.java files.*

***Logging Levels**: Can be adjusted to show more or less detail in the console, helping developers track the flow of data through the filters and services.*

| | | |

1. API Controller: UserController.java

The UserController acts as the entry point for all RESTful communication between the client (e.g., a mobile app or Postman) and the backend services.

Why is it used?

It separates the web-handling logic (HTTP status codes, URL mapping) from the core business logic, ensuring that the UserService remains independent of the web framework.

Key Endpoints Breakdown:

@GetMapping("/users"):

***Action**: Fetches a complete list of all users stored in the database.*

***Response**: Returns an HTTP 200 OK status along with the user list.*

@PostMapping("/user/save"):

***Action**: Receives a new User object and persists it to the database.*

***Logic**: It uses ServletUriComponentsBuilder to create a Location header in the response, which points to the newly created resource.*

@PostMapping("/role/addtouser"):

***Action**: Links an existing role to an existing user.*

***Input**: It accepts a RoleToUserForm DTO (Data Transfer Object) containing the username and roleName.*

2. Custom Exception: RoleNotFoundException.java

This class is a specialized exception that extends RuntimeException.

Why is it used?

In a production system, generic errors like "Internal Server Error" are difficult to debug. This custom exception provides specific feedback when a requested role is missing from the database, allowing for more precise error handling and logging.

Code Breakdown:

public RoleNotFoundException(String message):

***Action**: Accepts a custom string describing the error.*

***Logic**: It passes this message to the parent RuntimeException using super(message), which ensures the message is included in the application's stack trace and log files.*

3. Data Transfer Object: RoleToUserForm.java

A simple POJO (Plain Old Java Object) used to capture specific data from a JSON request body.

Why is it used?

Instead of sending a heavy User or Role entity across the network, this DTO contains only the two fields necessary to complete the "add role" operation: username and roleName.

Key Components:

***@Data**: A Lombok annotation that automatically generates getters and setters, allowing the Spring ObjectMapper to bind incoming JSON data to these fields.*

***Fields**: Consists of two private strings, username and roleName, which map directly to the keys expected in the frontend's POST request.*

1. Token Refresh Logic: UserController.java (Continued)

In a secure JWT architecture, access tokens are designed to expire quickly (e.g., 10 minutes) to minimize risk. The refreshToken method provides a way for users to stay logged in without re-entering credentials.

Why is it used?

*It allows the application to remain **stateless** while providing a seamless user experience. If an access token is compromised, it becomes useless after a short window, while the long-lived refresh token (stored securely) can be used to generate a new session.*

Key Logic Breakdown:

***Header Verification**: The method checks the Authorization header for a Bearer prefix, ensuring the request follows standard OAuth2 protocols.*

***Token Decoding**: It uses the same Algorithm.HMAC256("secret") used during login to verify the signature of the refresh token.*

***User Synchronization**: It extracts the username from the token and calls userService.getUser(username) to ensure the user still exists and hasn't had their roles changed since the last token was issued.*

***Response Generation**: If valid, a brand-new access_token is generated and sent back to the client in a JSON map.*

2. Automated Integrity Testing: UserServiceApplicationTests.java

This class contains the integration tests that validate the entire Spring Boot ecosystem.

Why is it used?

Testing ensures that changes to one part of the code (like the database schema) don't break another part (like the security filters). It acts as a safety net during the development lifecycle.

Key Components Breakdown:

***@SpringBootTest**:*

***What**: This annotation tells Spring to start the full application context, including the database connection and all security beans.*

***Why**: It confirms that the SecurityConfig, UserService, and UserRepo are all "wired" together correctly.*

***contextLoads()**:*

***What**: A basic "smoke test" that checks if the application can start without crashing.*

***Why**: If this test fails, it typically points to a critical configuration error in pom.xml or application.properties, such as a missing dependency or a failed database connection.*

3. Project Manifest: pom.xml (Review)

The pom.xml is the final piece that ties all these components together by managing the project's external libraries.

Why is it used?

Without this file, the Java compiler would not know how to handle JWTs, Spring Security, or the MySQL database. It defines the "DNA" of the project.

Core Dependencies Summary:

***java-jwt**: Specifically used by the CustomAuthenticationFilter and CustomAuthorizationFilter to create and verify tokens.*

***spring-boot-starter-data-jpa**: Powers the UserRepo and RoleRepo interfaces, allowing for seamless database interaction.*

***lombok**: Used across all entities and services to keep the code concise by automating the generation of constructors and loggers.*

1. Project Management: pom.xml

The pom.xml (Project Object Model) is the fundamental unit of work in Maven. It contains information about the project and configuration details used by Maven to build the project.

Why is it used?

It centralizes the management of all external libraries (dependencies), build plugins, and project metadata (like versioning), ensuring that any developer can build the project with identical results.

Key Components Breakdown:

***Project Metadata**: Defines the groupId (com.exmp), artifactId (userservice), and version (0.0.1-SNAPSHOT).*

***Spring Boot Starter Parent**: Inherits default configurations for Java versioning, UTF-8 encoding, and dependency management.*

***Dependencies**:*

***spring-boot-starter-web**: Provides the embedded Tomcat server and REST capabilities.*

***auth0 java-jwt**: The specific library used by the security filters to sign and verify tokens.*

***lombok**: Used to reduce boilerplate code in entities like User and Role.*

***Build Plugins**: Includes the spring-boot-maven-plugin, which allows the application to be packaged into a single executable JAR file.*

2. Environment Configuration: application.properties

This file located in src/main/resources holds the externalized configuration for the application.

Why is it used?

It allows the application to run in different environments (Development, Testing, Production) without changing the Java code. You can simply swap the values in this file to connect to a different database or change security settings.

Configuration Breakdown:

***Database Connectivity**: Contains the URL, username, and password for the MySQL database.*

***JPA/Hibernate Settings**:*

hibernate.ddl-auto: Controls how the database schema is synchronized with Java entities.

show-sql: Set to true to log the generated SQL queries to the console for debugging.

***Server Settings**: (Optional) Defines the port (default 8080) and context path for the API.*

3. Build Automation: mvnw and mvnw.cmd

These are the "Maven Wrapper" scripts for Unix-based systems and Windows, respectively.

Why are they used?

They allow a developer to run Maven commands (like ./mvnw clean install) without having Maven pre-installed on their machine.

Key Benefit:

***Version Consistency**: The wrapper ensures that everyone working on the project uses the exact same version of Maven defined in the .mvn/wrapper/maven-wrapper.properties file, preventing "it works on my machine" bugs during the build process.*

4. Version Control: .gitignore

A text file that tells Git which files or folders to ignore in a project.

Why is it used?
It prevents unnecessary or sensitive files from being uploaded to a repository.
What is ignored?
/target/*: The folder containing compiled .class files and the final JAR.*
IDE files*: Personal settings from IntelliJ (.idea/) or Eclipse.*
Sensitive data*: Local configuration files that might contain real passwords.*

1. Project Root & Build Wrapper: mvnw and mvnw.cmd

At the base of the UserService folder, you will find the Maven Wrapper scripts.

Why is it used?

The Maven Wrapper allows any developer or automated system (like a Jenkins CI/CD pipeline) to compile and build the project without having to manually install Maven. It ensures that the exact same version of the build tool is used across all environments, preventing version mismatch errors.

Key Files Breakdown:

***mvnw**: The shell script for Linux/macOS users.*

***mvnw.cmd**: The batch script for Windows users.*

***.mvn/wrapper**: Contains the small JAR and properties file that actually downloads the correct Maven version if it's missing.*

2. Dependency Management: pom.xml

The pom.xml (Project Object Model) file is the heart of the project's configuration.

Why is it used?

It acts as a centralized manifest that tells the Java compiler which external libraries are required to run the code. Instead of manually downloading JAR files, Maven fetches them automatically from a central repository.

Key Dependency Breakdown:

***spring-boot-starter-web**: Provides the foundational tools for building RESTful APIs, including the embedded Tomcat server.*

***spring-boot-starter-data-jpa**: Connects the Java code to the MySQL database, allowing for object-relational mapping.*

***java-jwt**: A specialized library from Auth0 used by the CustomAuthenticationFilter and CustomAuthorizationFilter to generate and verify secure tokens.*

***lombok**: An annotation-based library that reduces boilerplate code (like getters/setters) in the User and Role entities.*

3. Source Organization: src/main/java

The project follows the standard Maven directory structure to separate different concerns.

Why is it structured this way?

*Separating the code into specific packages (Controller, Service, Repository, Entity) follows the **Separation of Concerns** principle. This makes the code easier to navigate, test, and maintain.*

Directory Breakdown:

***controller/**: Handles incoming HTTP requests and routes them to the services.*

***service/**: Contains the "business logic"—the actual rules of how users and roles interact.*

***repository/**: The "Data Access Layer" that communicates directly with the database.*

***entity/**: Defines the data models that represent tables in the MySQL database.*

***security/**: Houses the JWT filters and the security configuration that protects the API.*


```
mysql> use userservice_db ;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> show tables
    -> ;
+--------------------------+
| Tables_in_userservice_db |
+--------------------------+
| role                     |
| role_seq                 |
| user                     |
| user_roles               |
| user_seq                 |
+--------------------------+
5 rows in set (0.002 sec)

mysql> SELECT * FROM user_roles;
+----------+---------+
| roles_id | user_id |
+----------+---------+
|        3 |       1 |
|        1 |       2 |
|        2 |       3 |
|        4 |       4 |
|        2 |       4 |
|        3 |       4 |
+----------+---------+
6 rows in set (0.001 sec)
```

http://localhost:8080/api/login

POST http://localhost:8080/api/login

Params Authorization Headers (9) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary

	Key	Value
☑	username	john
☑	password	1234
	Key	Value

ody Cookies (1) Headers (14) Test Results

Status: 200 OK Time: 130 ms Size: 835 B Save

Pretty Raw Preview Visualize JSON


```
1 {
2     "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
        eyJzdWIiOiJqb2huIiwiZXhwIjoxNzc0ODk3NzI2LCJpc3MiOiJodHRwOi8vbG9jYWxob3N0OjgwODAvYXBpL2xvZ2luIiwicm9sZXMiOlsiUk9MRV9VU0VSIl19.
        pzO0lf9iTeOr7gfbVVQQhemor37iu-hkZdUWoSSGkRQ",
3     "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJqb2huIiwiZXhwIjoxNzc3NDg5MTI2LCJpc3MiOiJodHRwOi8vbG9jYWxob3N0OjgwODAvYXBpL2xvZ2luIn0.
        yVBr9H49fQgQvvjsNnSa2pT6YT7Ouhyt-QC2zzF4k68"
4 }
```

http://localhost:8080/api/users

Save

GET http://localhost:8080/api/users Send

Params Authorization Headers (10) Body Pre-request Script Tests Settings Cookies

Headers 9 hidden

	Key	Value	Bulk Edit
☑	Authorization	Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJqb2huIiwiZXhwIjoxNzc0ODk3NTk4LC...	
	Key	Value	

Body Cookies (1) Headers (14) Test Results Status: 200 OK Time: 101 ms Size: 1.11 KB Save Response

Pretty Raw Preview Visualize JSON


```
[
    {
        "id": 1,
        "name": "John Traore",
        "userName": "john",
        "password": "$2a$10$tcswz4vW6ch9pLPTe11tK.jNzavaTnxRZxAFWdsoOx6TX6e8jEN4O",
        "roles": [
            {
                "id": 3,
                "name": "ROLE_USER"
            }
        ]
    },
    {
        "id": 2,
        "name": "Will Smith",
        "userName": "will",
        "password": "$2a$10$EQ.k27TSkfrd8putjsx0kuHiY2.0d35rDiS8fNzvbypICUJtGTdcq",
        "roles": [
            {
                "id": 1,
                "name": "ROLE_MANAGER"
            }
        ]
    },
    {
        "id": 3,
        "name": "Jim Carry",
        "userName": "jim",
        "password": "$2a$10$j2t3wOgv5TK/JB8tpuK9y.27XJj0Voa4o/bTC9wO.rCxXxBLFsKGu",
        "roles": [
```